Dummy synthesized attribute.

* If the semantic rule is in the form of a procedure call fun(a1, a2, a3, ... ak) then it can be transformed into the form b = fun(a1, a2, a3, ... ak) where b is a dummy synthesized attribute.
* If the semantic rule is associated with a procedure call, then there will be a dummy synthesized attribute.

Dependency graph:

* If an attribute b at a node in a parse tree depends on an attribute c, then the semantic rule for b at that node must be evaluated after the semantic rule that defines c.

c ↓ b

* The interdependencies among the inherited and synthesized attributes at the nodes in a parse tree can be depicted by a directed graph called a dependency graph.

→ The dependency graph for a given parse tree is constructed as follows:

for each node n in the parse tree do
for each attribute a of the grammar
symbol at n do
Construct a node in the dependency graph for
a
for each node n in the parse tree do
for each semantic rule $b = f(c_1, c_2, \cdots c_k)$
associated with the production used at
n do
for $i = 1$ to k do
construct an edge from the node for c_i
to the node for b

~~Construct~~
Consider,
<u>production</u>

$E \to E_1 + E_2$

<u>Semantic rule</u>

$E.val = E_1.val + E_2.val$

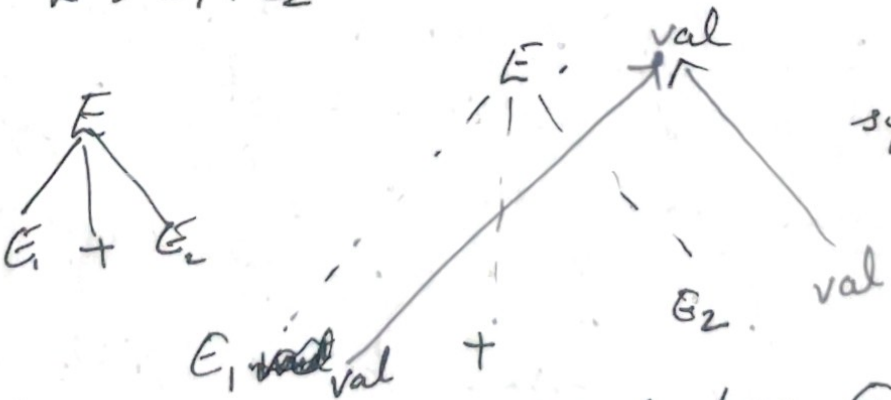


Fig. E.val is synthesized from $E_1.val$ and $E_2.val$.

The edge to E.val from $E_1.val$ shows that <u>E.val depends on $E_1.val$</u> and the edge to E.val ~~shows~~ from <u>$E_2.val$</u> shows that E.val ~~d~~ also depends on <u>$E_2.val$</u>.

* The dotted lines represent the parse tree and are not part of dependency graph.

* Consider ~~dep~~ syntax directed definitions that use inherited attributes are given below,

$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

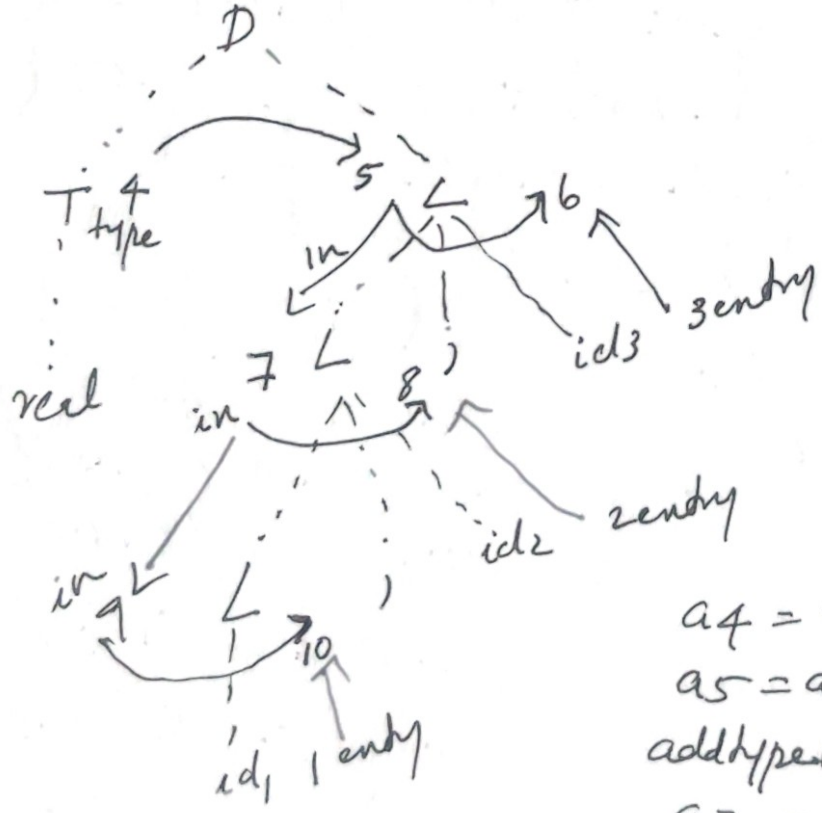


Fig. Dependency graph for parse trees.

$a_4 = real$
$a_5 = a_4$
$addtype(id_3.entry, a_5)$
$a_7 = a_5$
$addtype(id_2.entry, a_7)$
$a_9 = a_7$
$addtype(id_1.entry, a_9)$

Evaluating these semantic rules stores the type real in the symbol table.

L - attributed definitions

* A syntax directed definition is L-attributed, if each inherited attribute of X_j, $1 \le j \le n$ on the right side of $A \to X_1 X_2 \cdots X_n$, depends on only on

① the attributes of the symbols $X_1, X_2, \cdots X_{j-1}$ to the left of X_j in the production and

② the inherited attributes of A.

* Every S-attributed definition is L-attributed, because the restrictions (1) and (2) apply to inherited attributes only.

Consider the example,

Production	Semantic rule.
$A \to LM$	$L.i = l(A.i)$ $M.i = m(L.s)$ $A.s = f(M.s)$
$A \to QR$	$R.i = r(A.i)$ $Q.i = q(R.s)$ $A.s = f(Q.s)$

The second one is non L-attributed syntax directed definition.

Translation Schemes

* It is a context free grammar, in which attributes are associated with the grammar symbols and semantic actions enclosed between braces { } are inserted within the right side of productions.

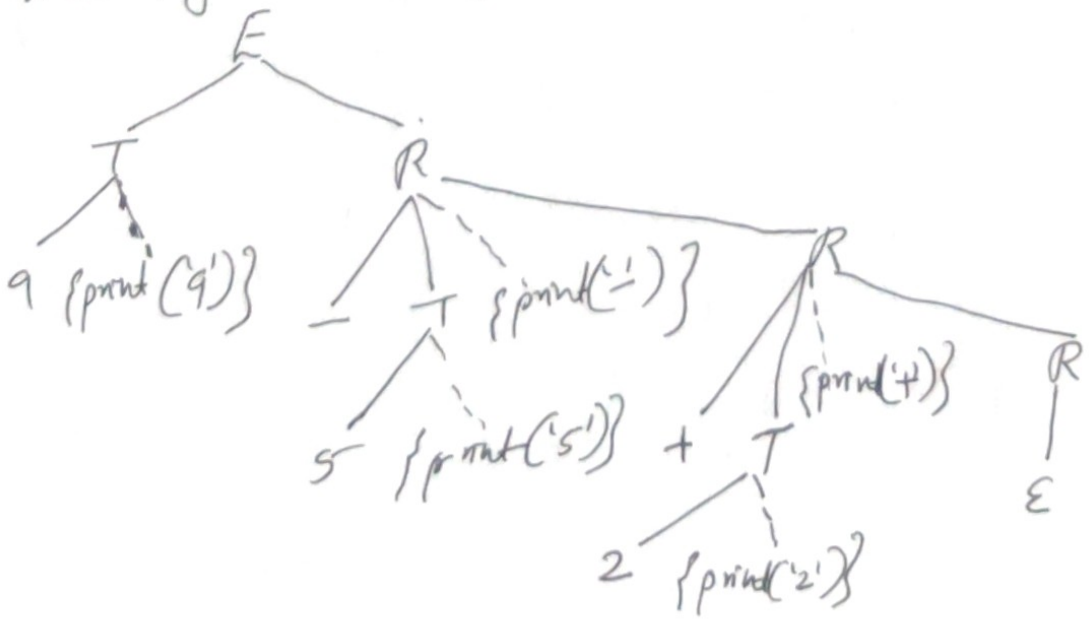
* A simple translation scheme that maps infix expressions with addition and subtraction into the corresponding postfix expressions are given below:

$E \rightarrow TR$

$R \rightarrow addop\ T\ \{print(addop.lexeme)\}\ R_1 \mid \varepsilon$

$T \rightarrow num\ \{print(num.val)\}$

Fig. shows the parse tree for the input 9−5+2 with each semantic action attached as the appropriate child of the node corresponding to the left side of their production.



* When performed in depth-first order, the actions in fig. 5.20 print the o/p 95-2+.

Bottom-up Evaluation of S-attributed Definitions

* Synthesized attributes can be evaluated by a bottom-up parser as the input is being parsed.
* The parser can keep the values of the synthesized attributes associated with the grammar symbols in its stack.
* Whenever a reduction is made, the values of the new synthesized attributes are computed from the attributes appearing on the stack for the grammar symbols on the right side of the reducing production.
* A translator for an S-attributed definition can be implemented with the help of an LR parser generator.
* A bottom up parser uses a stack to hold information about subtrees that have been parsed.